Software Requirements Specification

for

Concurrent File Sharing System (TCP)

Version 1.1 updated

Prepared by Team 19

Team 19, Computer Networks Lab

11 November 2026

Table of Contents

Table of Contents **2**
Revision History **2**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description **3**
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 4
2.7 Assumptions and Dependencies 5
3. External Interface Requirements **5**
3.1 User Interfaces 5
3.2 Hardware Interfaces 5
3.3 Software Interfaces 5
3.4 Communications Interfaces 5
4. System Features **7**
5. Other Nonfunctional Requirements **9**
5.1 Performance Requirements 9
5.2 Safety Requirements 10
5.3 Security Requirements 10
5.4 Software Quality Attributes 10
5.5 Business Rules 10
6. Other Requirements **10**
Appendix A: Glossary **10**
Appendix B: Analysis Models **11**
Appendix C: To Be Determined List ..... Error! Bookmark not defined.

Revision History

Name	Date	Reason For Changes	Version
Team 19	11 November 2026	Updated to reflect POX/OpenFlow 1.0 controller, OVS HTB queue policies, runtime host controls and bandwidth monitoring.	1.1

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Concurrent File Sharing System (TCP), version 1.1, developed by Team 19. The system is a client-server application in which a multi-threaded TCP server lets several authenticated clients list, upload and download files concurrently, with SHA-256 integrity verification. It is evaluated on a Mininet network using an Open vSwitch (OVS) switch controlled by a POX-based SDN controller through OpenFlow 1.0. The controller applies per-host forwarding, blocking and bandwidth policies using OVS queues.

This SRS covers the complete product: the TCP server, command-line client, server-side storage, Mininet topology, OVS switch, custom POX controller component, runtime network policies and bandwidth/performance measurement tools.

1.2 Document Conventions

Requirements are identified as REQ-<feature>.<number> and each is a single, verifiable statement. The words "shall" (mandatory) and "should" (desirable) are used deliberately. Every requirement inherits the priority of its feature unless a priority is stated. Commands, file names, JSON fields and code identifiers appear in a monospaced style where possible (for example LOGIN, users.json and csfs_pox.py).

1.3 Intended Audience and Reading Suggestions

Intended readers: course instructors and evaluators (what the system must do), developers (Sections 3 to 5), testers (Section 4 requirements and Section 5 targets for the test plan), and project teammates maintaining the code.

Suggested reading order: Sections 1 and 2 for context, Section 3 for interfaces and the wire protocol, Section 4 for system features, and Section 5 for non-functional requirements. The companion Software Architecture Document (SAD) explains how these requirements are realised.

1.4 Product Scope

The product lets a group of users share files through one central server over TCP. Its goals are: (1) correct and complete file transfer verified by SHA-256; (2) concurrent client sessions so one stalled client does not block others; (3) role-based access control for regular users and administrators; (4) SDN-based host-level forwarding, blocking and bandwidth policies using POX, OpenFlow 1.0 and OVS HTB queues; and (5) measurable behaviour under controlled Mininet conditions, including per-host bandwidth monitoring and SDN/non-SDN performance experiments.

1.5 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- K. Wiegers, SRS template (the template this document follows).
- Team 19, Software Architecture Document (SAD), updated version 1.1 (to be aligned with this SRS).
- Python 3 socket, threading, hashlib, struct, json and subprocess library documentation (python.org).

- Mininet, Open vSwitch and POX documentation; OpenFlow 1.0 references.

2. Overall Description

2.1 Product Perspective

The product consists of the TCP file server (src/server.py), command-line client (src/client.py), Mininet topology (topology/topo.py), OVS switch s1 and a POX controller process. The custom POX component (controller/csfs_pox.py) communicates with OVS over OpenFlow 1.0 and configures per-host OVS HTB queues. The file server sends LOGIN, LOGOUT and FAILS events to the controller over the local Unix domain socket /tmp/sdn_ctl.sock. The controller also accepts runtime operator commands through this socket.

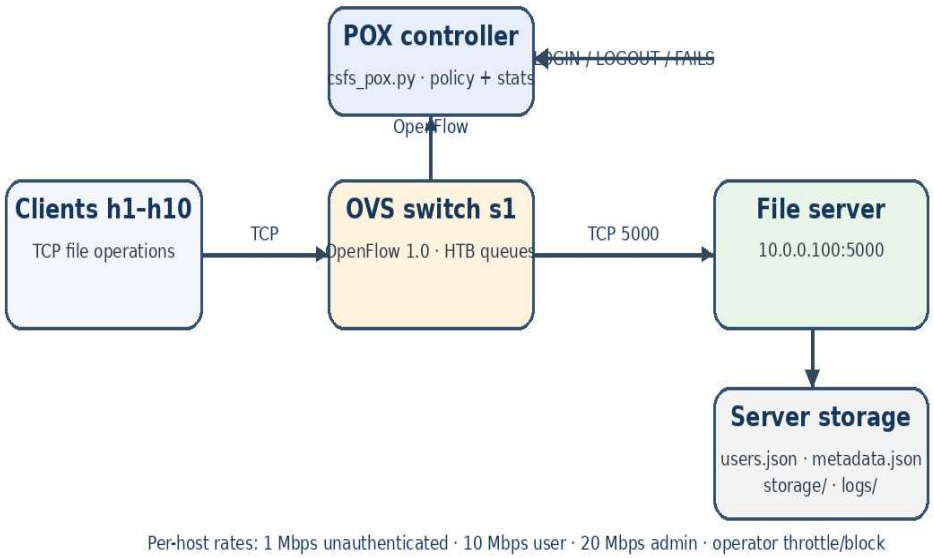


Figure 1: Updated logical architecture. TCP clients communicate with the file server through OVS s1; POX controls OVS using OpenFlow 1.0, while the server sends authentication events to POX over a local Unix socket.

2.2 Product Functions

- *Authenticate users and assign user or admin roles.*
- List files that the authenticated user is permitted to access.
- Upload files after size/name validation and SHA-256 integrity checking.
- Download files and verify integrity on the client using SHA-256.
- Serve multiple client sessions concurrently using one thread per connection.
- Record significant server and controller events and maintain per-host bandwidth measurements.
- Use POX/OpenFlow 1.0 to install OVS forwarding/drop rules, apply per-host bandwidth limits, support runtime operator policies and compare SDN with a non-SDN baseline.

2.3 User Classes and Characteristics

User class	Characteristics	Privileges
Regular user (alice, bob)	Technical student; uses the command-line client; frequent use.	Log in; list, upload and download own files.
Administrator (admin)	Same skills; occasional use for oversight and	All user functions, plus list and download

	testing.	every file.
Network operator / tester	Runs Mininet and POX; starts topology/controller, performs experiments and reviews logs/CSV results.	Starts server/topology/controller; checks status/flows; blocks, unblocks, throttles and unthrottles hosts; reviews bandwidth and performance results.

Regular users and administrators use the command-line client. The network operator/tester starts Mininet and POX, manages host policies and runs performance experiments.

2.4 Operating Environment

Ubuntu/Linux environment with Python 3, Mininet 2.3.0, Open vSwitch (README identifies 3.7.1), POX 0.7.0 cloned separately into the project folder, and python3-matplotlib for graph generation. POX is not included in the repository ZIP. The README notes a Python 3.14 compatibility warning for POX 0.7.0; the exact Python/POX compatibility must be verified on the final evaluation machine.

2.5 Design and Implementation Constraints

- *The client, server, topology and custom controller component are implemented in Python 3; POX is an external dependency.*
- Transport between client and server must be TCP; control data uses a length-prefixed JSON header and file data is sent as raw bytes.
- The server handles each accepted client connection in a separate thread.
- The SDN topology uses Mininet and OVS; SDN mode uses POX with OpenFlow 1.0. The non-SDN baseline runs without POX.
- OpenFlow 1.0 does not provide the meter feature used by later OpenFlow versions. Rate limits are implemented using OVS HTB queues configured through ovs-vsctl. Traffic is not encrypted.

2.6 User Documentation

The README.md provides setup, launch, operator-command and experiment instructions. docs/PROTOCOL.md describes the TCP protocol and server-to-controller events. This SRS, the SAD and the IEEE-style Test Plan form the project documentation set.

2.7 Assumptions and Dependencies

- *Assumption: the demonstration network is a controlled lab network; TCP traffic and credentials are not encrypted.*
- Assumption: Mininet client IP addresses identify hosts for the controller's policy mapping.
- Assumption: the file metadata index is small enough to be loaded into memory for each operation.
- Dependency: Mininet and Open vSwitch are installed and run with the required privileges.
- Dependency: POX is cloned into the project folder and can run with the selected Python interpreter; the server file system has sufficient free space.

3. External Interface Requirements

3.1 User Interfaces

The user interface is a command-line client invoked in the general form: python3 src/client.py --host HOST --port PORT -u USER -p PASSWORD [--out DIR] ACTION [ARG], where ACTION is list, upload or download. The client prints operation results and transfer time/throughput. Login failure exits with code 2, a general operation error exits with code 1 and a download integrity failure exits with code 3. The controller launcher supports starting POX and sending runtime commands using controller/controller.py --send "COMMAND".

3.2 Hardware Interfaces

No special hardware is used. The product relies on a standard network interface (virtual interfaces veth/OVS in Mininet) and the local disk.

3.3 Software Interfaces

- Operating system: Linux sockets (TCP/IPv4).
- Python 3 libraries: socket, threading, json, struct, hashlib, secrets, os, csv, subprocess and related standard-library modules.
- Mininet 2.3.0, Open vSwitch and POX 0.7.0 (external clone) for topology, switching and SDN control.
- Local files: users.json (accounts), metadata.json (file index), storage/ (file contents), logs/server.log and results/bandwidth_log.csv plus performance result files.

3.4 Communications Interfaces

Client-server communication uses one TCP connection per client session to the server (default port 5000). Each control message is a 4-byte unsigned big-endian length followed by a UTF-8 JSON object (maximum 1 MiB). File content is sent as raw bytes in chunks of up to 64 KiB. In SDN mode, the server sends JSON-line LOGIN, LOGOUT and FAILS events to the controller through the Unix domain socket /tmp/sdn_ctl.sock. The controller communicates with OVS s1 using OpenFlow 1.0 on port 6633. OVS queues are configured using ovs-vsctl; per-host flow statistics are polled approximately once per second.

Command	Request fields	Response
LOGIN	user, password	status ok + role, or error
LIST	(none)	status ok + files[] (name, owner, size, sha256, uploaded)
UPLOAD	name, size, sha256	ready (then raw bytes), then ok + sha256 or error
DOWNLOAD	name	ok + size + sha256, then raw bytes; or error
QUIT	(none)	connection closed
POX	Python-based SDN controller framework hosting the team's custom csfs_pox component.	
OpenFlow 1.0	Southbound protocol used by POX to control OVS in SDN mode.	
HTB queue	Hierarchical Token Bucket queue used by OVS QoS to enforce per-host rate limits.	
Unix domain socket	Local inter-process communication used for server-controller events and operator commands.	

4. System Features

The features below are organised by major service. Priorities are High (H), Medium (M) or Low (L). The network-control and measurement features are Medium priority.

4.1 User Authentication

4.1.1 Description and Priority

Users identify themselves with a user name and password before any file operation. Passwords are stored as salted SHA-256 hashes. The server reports successful login, logout and repeated failed-login events to the POX controller so that the host's network policy can be updated. Priority: High.

4.1.2 Stimulus/Response Sequences

- Client sends LOGIN with user and password.
- Server replies ok with the user's role, or an error "invalid credentials".
- After 3 failed attempts on one connection the server closes it; after 3 failed attempts from an IP address, the server notifies the controller, which blocks that host for 30 seconds.

4.1.3 Functional Requirements

REQ-1.1: The server shall reject every command other than LOGIN until the session is authenticated, replying "login required".

REQ-1.2: The server shall compare a salted SHA-256 hash of the supplied password with the stored hash using a constant-time comparison.

REQ-1.3: The server shall close the connection after 3 consecutive failed login attempts.

REQ-1.4: The server shall record each successful and failed login, with client address, in the log, without recording plaintext passwords.

REQ-1.5: If users.json does not exist, the system shall create the demonstration accounts alice and bob (role user) and admin (role admin), with salted password hashes.

4.2 File Listing

4.2.1 Description and Priority

An authenticated user obtains the names and details of the files they are allowed to access. Priority: High.

4.2.2 Stimulus/Response Sequences

- Client sends LIST.
- Server replies with the accessible files and their owner, size, SHA-256 and upload time.

4.2.3 Functional Requirements

REQ-2.1: The server shall return only files owned by the requesting user when the role is user.

REQ-2.2: The server shall return all files when the role is admin.

REQ-2.3: The server shall return an empty list, not an error, when no files are accessible.

4.3 File Upload

4.3.1 Description and Priority

A user stores a file on the server. The client sends the file size and SHA-256 first; the server stores the data only if the received bytes match. Priority: High.

4.3.2 Stimulus/Response Sequences

- Client sends UPLOAD with name, size and sha256.
- Server replies ready (or an error).
- Client streams the bytes; server replies ok or an integrity error.

4.3.3 Functional Requirements

REQ-3.1: The server shall reject an upload whose name is empty, is "." or "..", or starts with "."; any directory part of a name shall be discarded.

REQ-3.2: The server shall reject an upload larger than 1 GiB or with a negative or non-numeric size.

REQ-3.3: The server shall write incoming data to a temporary file and move it into storage only after the SHA-256 matches the value declared by the client.

REQ-3.4: The server shall delete the temporary file and keep any previous version if the transfer is interrupted or the checksum does not match.

REQ-3.5: The server shall refuse to overwrite a file owned by another user unless the requester is an admin.

REQ-3.6: The server shall update metadata.json atomically and record owner, size, SHA-256 and upload time.

4.4 File Download

4.4.1 Description and Priority

A user retrieves a file they are allowed to access and verifies it against the stored checksum. Priority: High.

4.4.2 Stimulus/Response Sequences

- Client sends DOWNLOAD with a file name.
- Server replies ok with size and sha256, then streams the bytes.
- Client computes SHA-256 and reports INTEGRITY OK or INTEGRITY FAILED.

4.4.3 Functional Requirements

REQ-4.1: The server shall reply "file not found" for an unknown name and "permission denied" when the user is neither owner nor admin.

REQ-4.2: The server shall send the stored SHA-256 together with the file size before the data.

REQ-4.3: The client shall verify the received data against that SHA-256 and exit with code 3 on mismatch.

REQ-4.4: The client shall report transfer time and throughput in Mbps for uploads and downloads.

4.5 Concurrent Sessions

4.5.1 Description and Priority

Multiple clients can be connected and transferring at the same time. Priority: High.

4.5.2 Stimulus/Response Sequences

- A client opens a TCP connection to the server port.
- The server accepts it and starts an independent handler.

4.5.3 Functional Requirements

REQ-5.1: The server shall handle each connection in its own thread so that one client cannot block another.

REQ-5.2: The server shall protect shared state (metadata file and log) with locks so that concurrent updates are not lost or interleaved.

REQ-5.3: The server shall support at least 10 simultaneous clients (the number of hosts in the test topology).

REQ-5.4: The server shall close an idle or stalled connection after 120 seconds and release its resources.

4.6 Logging

4.6.1 Description and Priority

The server keeps an audit trail of connections and file operations. Priority: Medium.

4.6.2 Stimulus/Response Sequences

- Any connect, login, list, upload, download, denial, integrity failure or disconnect occurs.

4.6.3 Functional Requirements

REQ-6.1: The server shall append a timestamped line to logs/server.log and to the console for each such event.

REQ-6.2: Log lines shall not contain passwords.

4.7 Network Policy (Mininet/OVS)

4.7.1 Description and Priority

In the emulated network, OVS switch s1 is controlled by a custom POX component using OpenFlow 1.0. The controller installs forwarding and drop rules and configures per-host HTB queues through ovs-vsctl. It applies role-based rates, temporary blocking and operator-defined runtime policies. Priority: Medium.

4.7.2 Stimulus/Response Sequences

- The operator starts the Mininet topology and POX controller; the switch connects to POX over OpenFlow 1.0.
- The controller installs base forwarding rules and queue policies. Server LOGIN/LOGOUT/FAILS events and operator commands update the corresponding host policy at runtime.

4.7.3 Functional Requirements

REQ-7.1: The topology shall contain one OVS switch s1, one server host (10.0.0.100) and the configured client hosts (10.0.0.1 and up; default ten clients).

REQ-7.2: In SDN mode, OVS shall connect to the POX controller using OpenFlow 1.0. The controller shall install a default-deny rule, an ARP forwarding rule and per-host TCP port 5000 forwarding rules to/from the server.

REQ-7.3: The controller shall apply per-host OVS HTB queue limits: 1 Mbps for unauthenticated hosts, 10 Mbps for user-role hosts and 20 Mbps for admin-role hosts. The default operator throttle rate shall be 2 Mbps; the operator may specify another rate.

REQ-7.4: The controller shall block a host for 30 seconds after receiving a FAILS event for three failed logins, support operator commands status, flows, block, unblock, throttle and unthrottle, and collect per-host flow statistics approximately every second in results/bandwidth_log.csv. A non-SDN baseline mode shall also be available.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Targets below are stated so they can be measured by the Test Plan.

- The default topology shall support experiments with up to 10 client hosts transferring files concurrently; the supplied hard-scenario result records eight surviving transfers completing successfully after one client is interrupted.
- The supplied performance results show approximately 9.4–9.6 Mbps per client under the 10 Mbps SDN user queue in the tested VM; actual throughput depends on link capacity and environment.
- The supplied queue test records approximately 4.7–4.8 Mbps for a 5 Mbps queue, 18.4 Mbps for a 20 Mbps queue and 0.96 Mbps for a 1 Mbps queue. These are experimental observations, not guaranteed limits.
- The controller should produce per-host bandwidth samples at approximately one-second intervals. The Test Plan shall define the acceptance tolerance for configured versus measured throughput.

5.2 Safety Requirements

Uploads are written to a temporary file and committed only after the received bytes match the declared SHA-256 digest. An interrupted or integrity-failed upload removes its temporary file. Important files should also be retained outside the server because the system does not provide backup or recovery.

5.3 Security Requirements

- Authentication is required for all file operations (REQ-1.1).
- Passwords are stored as salted SHA-256 hashes and never logged.
- Authorisation: owner or admin only (REQ-2.1, 2.2, 3.5, 4.1).
- Path traversal is prevented by name sanitisation (REQ-3.1).
- Header size (1 MiB) and file size (1 GiB) are bounded to limit resource abuse.
- Known limitation: traffic is not encrypted and passwords are sent in clear text over TCP and supplied as command-line arguments. The system shall be used only on a trusted lab network. SHA-256 is not a password-stretching function.

5.4 Software Quality Attributes

- Correctness: transferred files are bit-identical to the original (verified by SHA-256).
- Reliability: a client failure or interrupted transfer does not stop the server or corrupt stored data.
- Maintainability: server, client, topology, controller launcher and POX component are separated into repository folders.
- Portability: the file-sharing application targets Linux with Python 3; SDN experiments additionally require compatible Mininet, OVS and POX installations.
- Testability: core behaviour is checked through command-line clients, runtime policy tests, hard-scenario tests, queue tests and performance scripts; outputs include logs, CSV files and graphs.
- Observability: server/controller logs, per-host bandwidth CSV records and performance summaries provide evidence of system behaviour.

5.5 Business Rules

- A user may access only files they own.
- An administrator may list and download every file; upload overwrite is permitted only where the server's ownership check allows it (including admin role).
- File names are unique across the whole server (a flat namespace).

6. Other Requirements

Data storage: accounts are stored in users.json, a file index in metadata.json and file contents in storage/. Server events are recorded in logs/server.log. Controller bandwidth measurements and experiment outputs are stored under results/ (including bandwidth_log.csv, perf_raw.csv and perf_summary.csv). No database server is used. Internationalisation and legal requirements are out of scope.

Appendix A: Glossary

Term	Meaning
TCP	Transmission Control Protocol; reliable, ordered byte stream.
SHA-256	Cryptographic hash used to check file integrity.
OVS	Open vSwitch, the virtual switch used in Mininet.
Mininet	Network emulator used to build the test topology.
Thread per client	Concurrency model with one OS thread for each connection.
Frame	4-byte length followed by a JSON message.
SRS / SAD	Software Requirements Specification / Software Architecture Document.

Appendix B: Analysis Models

Use cases: Log in; List files; Upload file; Download file; Verify upload/download integrity; Notify the SDN controller of LOGIN, LOGOUT and FAILS events; Configure network policy (operator); Monitor per-host bandwidth; Run SDN/non-SDN performance experiments. Session state: Connected, then Authenticated (after LOGIN ok), then Closed (QUIT, 3 failed login attempts on the connection, error or 120 s timeout). Controller host states include unauthenticated, user, admin, throttled and blocked.

Appendix C: To Be Determined List

- *TBD-1: Exact acceptance tolerance for measured throughput versus configured OVS queue rate (to be fixed in the Test Plan).*
- TBD-2: Confirm Python/POX 0.7.0 compatibility on the final evaluation machine. TLS and stronger password hashing remain outside version 1.1 scope.